API & Integration Guide — Ultron-Qwen3.5-2B-ToolCall

1. Model Artifact

- **File**: `ultron-qwen3.5-2b-toolcall.Q6_K.gguf`
- **Quantization**: Q6_K (~6.5 bits/weight, near-FP16 quality, llama.cpp family)
- **Recommended runtime**: `llama.cpp` server, Ollama, or LM Studio (all have first-class GGUF + function-calling support)

2. Local Serving Options

Option A — llama.cpp server

```
./llama-server \
  -m ultron-qwen3.5-2b-toolcall.Q6_K.gguf \
  --host 0.0.0.0 --port 8080 \
  --ctx-size 32768 \
  --jinja   # enables chat template + tool-calling formatting
```

Exposes an OpenAI-compatible `/v1/chat/completions` endpoint, including `tools` and `tool_choice`.

Option B — Ollama

```
ollama create ultron-toolcall -f Modelfile
ollama run ultron-toolcall
```

`Modelfile` should reference the GGUF path and set the appropriate chat template / context size.

3. Request Format (OpenAI-compatible)

```
{
  "model": "ultron-toolcall",
  "messages": [
    { "role": "system", "content": "You are Ultron, a scoped pentesting assistant. Only call tools listed. Always confirm before high-risk actions." },
    { "role": "user", "content": "Start recon on the engagement's web server." }
  ],
  "tools": [ /* tool schemas from TOOL_SCHEMA_REFERENCE.md */ ],
  "tool_choice": "auto",
  "temperature": 0.2
}
```

4. Response Handling

The model returns a `tool_calls` array (name + JSON arguments) rather than raw text when it decides to invoke a tool. The orchestration layer must:

1. **Validate** the call against the current engagement scope (`engagement_id`, `authorized_targets`).
2. **Gate** any call flagged `risk_level: medium|high` behind `request_human_confirmation` before execution.
3. **Execute** the underlying tool/script and capture output.

4. **Feed the tool result back** to the model as a `tool` role message so it can continue the conversation/plan.
5. **Log** every call, argument set, and result for the audit trail (see `RESPONSIBLE_USE_POLICY.md`).

5. Deployment Requirements

Resource	Requirement
VRAM (GPU)	~2-3 GB for Q6_K at 2B parameters, plus context-size overhead
CPU-only	Feasible on a modern laptop CPU; expect lower throughput than GPU
RAM	8 GB+ recommended for CPU inference with headroom for the orchestration layer
Disk	GGUF file size (Q6_K, 2B) is roughly 1.5-2 GB
Network	None required for inference — supports fully offline / air-gapped engagements

6. Orchestration Layer Responsibilities

The model is a *decision-maker*, not an *executor*. The surrounding application must own:

- Engagement/scope registration and enforcement
- Tool execution sandboxing
- Human-in-the-loop confirmation UI
- Full audit logging (who approved what, when)
- Rate limiting / kill switch for runaway tool-call loops

7. Known Integration Gaps (to close before demo)

- [ ] No automated test harness yet confirming the model rejects out-of-scope targets
- [ ] No held-out integration test suite for multi-turn tool chains
- [ ] Confirmation UI for `request_human_confirmation` is `[not yet built / built — update this line]`